32、解决32M spi-nor无法复位问题

全志V3S嵌入式驱动开发（解决32M spi-nor无法复位问题）

原创

嵌入式-老费Open: 7f185b501ef38a059b10d1b79d7cfdb9_MD5.png

已于 2023-08-29 15:21:30 修改

Open: 8a923f94c34ac151897c6be747452d7c_MD5.png

阅读量697 Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏 2

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

摘要 本文介绍了全志V3S在使用SPI-NOR启动时遇到的无法复位问题，通过查阅资料和修改驱动代码解决了该问题。首先在论坛找到了可能的原因，即4 byte访问模式导致的复位失败。接着，尝试按照建议修改w25p80.c文件，在芯片移除时发送0x66和0x99命令以复位。然后，由于内核版本不匹配，进行了适配性调整。最终在新版本内核中实现了模式切换功能，成功解决了复位问题。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing @163.com】

之前制作spi-nor image的时候，就发现v3s存在无法复位的问题。只要进入linux之后，不管是console输入reboot指令，还是按下复位键，都存在无法重启的问题。一开始，以为是硬件哪里出了问题，但是sd卡启动或者spi-nand启动的时候，则没有这个现象。这说明，应该不是硬件的问题，或者至少说不是共性问题，需要个别好好研究一下。

1、第一步，论坛找方案

出现问题之后，第一步就是上网看看有没有其他同学遇到类似的问题，果然还发现了一个，

```
https://whycan.com/t_4550.html
```

在它的描述中，输入reboot没有重启，主要是因为spi-nor还处于4 byte访问模式，没有办法重新启动。如果想reboot之后，还能正常从spi-nor启动，就必须要在启动前退出4 byte模式。之前选用spi-nor的时候，挑选的是mx25l25645g，大小是256M bit，即32M byte，确实超过了16M的正常地址访问空间。

论坛上的讨论还说到，如果想退出这个模式，要么手改驱动，要么换上可以置位的pmu。当然，换pmu太麻烦了，还是修改驱动代码吧。网页上也给出了对应的地址，

```
https://whycan.com/t_534.html#p1447
```

不想查看网页的朋友，可以直接看这个修改补丁，

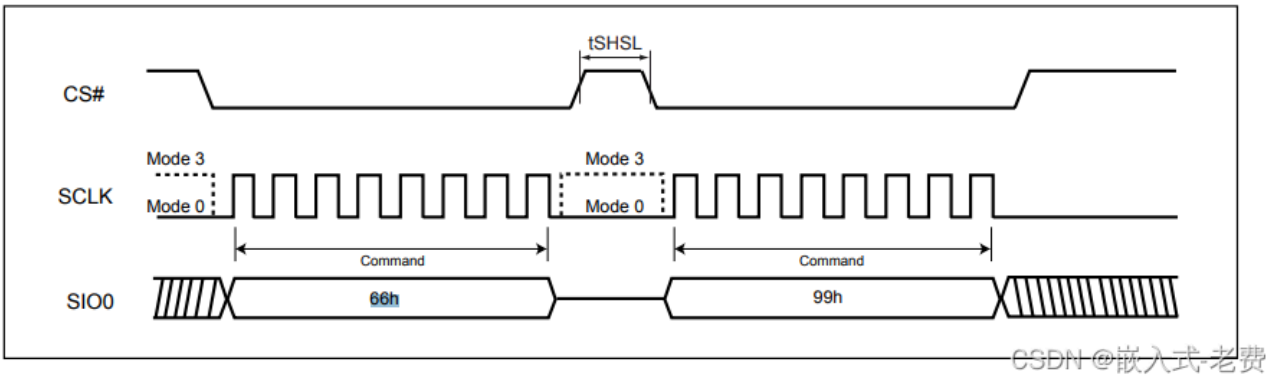
```
static int m25p_remove(struct spi_device *spi)
{
    struct m25p *flash = spi_get_drvdata(spi);

    // manfeel note: add spi flash reset code
    flash->command[0] = 0x66;
    spi_write(flash->spi, flash->command, 1);
    flash->command[0] = 0x99;
    spi_write(flash->spi, flash->command, 1);
    /* Clean up MTD stuff. */
    return mtd_device_unregister(&flash->mtd);
}


static struct spi_driver m25p80_driver = {
    .driver = {
        .name   = "m25p80",
        .owner  = THIS_MODULE,
    },
    .id_table   = m25p_ids,
    .probe  = m25p_probe,
    .remove = m25p_remove,
    // manfeel, add shutdown method to reset spi flash
    .shutdown = m25p_remove,

    /* REVISIT: many of these chips have deep power-down modes, which
     * should clearly be entered on suspend() to minimize power use.
     * And also when they're otherwise idle...
     */
};
```

修改的文件位于w25p80.c，修改也确实不复杂，主要就是在芯片remove的时候发送两条命令，一条是0x66，一条是0x99。为了知道这两条命令啥意思，我们特定找了

mx25l25645g的芯片手册来看，

Open: 9758f2753aa9b387d29fae09e3e08ad1_MD5.png

Figure 87. Reset Sequence (SPI mode)



它的中心意思还是比较简单。一句话来说，就是实现spi-nor的复位模式，用0x66、0x99来实现和断电复位一样的效果。

2、准备测试

看上去一切都准备好了。可是还是出错了，问题就出在我们的kernel比较新，是linux kernel 5.2.y版本，编译出来一堆错误，根本没有办法编译通过。比如它会告诉你，flash没有spi这个变量，spi没有command这个变量等等，这些都很明显，就是kernel版本不匹配，

```
drivers/mtd/devices/m25p80.c: In function ‘m25p_remove’:
drivers/mtd/devices/m25p80.c:249:7: error: ‘struct m25p’ has no member
named ‘command’
  flash->command[0] = 0x66;
       ^~
drivers/mtd/devices/m25p80.c:250:19: error: ‘struct m25p’ has no member
named ‘spi’; did you mean ‘spimem’?
  spi_write(flash->spi, flash->command, 1);
                  ^~~
                  spimem
drivers/mtd/devices/m25p80.c:250:29: error: ‘struct m25p’ has no member
named ‘command’
  spi_write(flash->spi, flash->command, 1);
                              ^~
drivers/mtd/devices/m25p80.c:251:14: error: ‘struct m25p’ has no member
named ‘command’
         flash->command[0] = 0x99;
              ^~
```

```
drivers/mtd/devices/m25p80.c:252:26: error: ‘struct m25p’ has no member
named ‘spi’; did you mean ‘spimem’?
         spi_write(flash->spi, flash->command, 1);
                           ^~~
                           spimem
drivers/mtd/devices/m25p80.c:252:36: error: ‘struct m25p’ has no member
named ‘command’
         spi_write(flash->spi, flash->command, 1);
                                       ^~
make[3]: *** [scripts/Makefile.build:279: drivers/mtd/devices/m25p80.o]
Error 1
make[2]: *** [scripts/Makefile.build:498: drivers/mtd/devices] Error 2
make[1]: *** [scripts/Makefile.build:498: drivers/mtd] Error 2
make: *** [Makefile:1077: drivers] Error 2
```

3、另辟蹊径

到这里，有的同学也许会说，既然这样，我们不如研究一下如何在现有版本的基础上实现spi_write这些内容。可人毕竟是想偷懒一下的，总想找到一个更容易的办法。很幸运，我们又找到这个链接，

```
https://blog.csdn.net/Adrian503/article/details/118484202
```

文章提示我们，现在新版本的kernel已经支持spi-nor在reboot的时候，进行模式切换了。它所要做的也很简单，就是在spi-nor.c文件中，把对应型号的模式修改一下即可，这很对我们的脾气。修改前，驱动的注册是这样的，

```
{ "mx25l25645g", INFO(0xc22019, 0, 64 * 1024, 512, 0) },
```

修改之后，是这样的，

```
{ "mx25l25645g", INFO(0xc22019, 0, 64 * 1024, 512, SPI_NOR_4B_OPCODES)
},
```

修改后，验证了一下，果然是有效的。不管是手动输入reboot，还是按下复位键，系统都可以正常重启了。这说明我们的修改是对的。

4、修改过程中的小插曲

这次因为修改的是spi-nor镜像，所以一部分代码又从spi-nand回退过来了。但是回退的过程中就发现了，我们修改了kernel代码之后，内核没有办法自动从uboot加载了。这还不是最诡异的，最奇怪的是手动sf read和bootz之后却又是可以加载的，只不过文件系统又起不来了，真是匪夷所思。

```
[    7.166496] VFS: Mounted root (jffs2 filesystem) on device 31:3.
[    7.172557] devtmpfs: error mounting -2
[    7.177734] Freeing unused kernel memory: 1024K
[    7.182449] Run /sbin/init as init process
[    7.186654] Run /etc/init as init process
[    7.190678] Run /bin/init as init process
[    7.194754] Run /bin/sh as init process
[    7.198602] Kernel panic - not syncing: No working init found.  Try
passing init= option to kernel. See Linux Documentation/admin-
guide/init.rst for guidance.
[    7.212754] CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.2.0-
licheepi-zero #14
[    7.219877] Hardware name: Allwinner sun8i Family
[    7.224611] [<c010ede4>] (unwind_backtrace) from [<c010b774>]
(show_stack+0x10/0x14)
[    7.232353] [<c010b774>] (show_stack) from [<c075c598>]
(dump_stack+0x88/0x9c)
[    7.239572] [<c075c598>] (dump_stack) from [<c011dd64>]
(panic+0x110/0x2fc)
[    7.246534] [<c011dd64>] (panic) from [<c0772dbc>]
(kernel_init+0xfc/0x10c)
[    7.253494] [<c0772dbc>] (kernel_init) from [<c01010e8>]
(ret_from_fork+0x14/0x2c)
[    7.261051] Exception stack(0xc3833fb0 to 0xc3833ff8)
[    7.266099] 3fa0:                                     00000000
00000000 00000000 00000000
[    7.274266] 3fc0: 00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000
[    7.282431] 3fe0: 00000000 00000000 00000000 00000000 00000013
00000000
[    7.289051] Rebooting in 5 seconds..
```

问题折腾来折腾去，弄了一个晚上，直到后来休息了一会，才想起来应该是自己把spi-nand的布局模式当成了之前spi-nor的布局。spi-nand的布局中，0x100000 ~0×120000是dtb，0×120000 ~ 0×620000是kernel。而spi-nor的布局中，

0x100000~0×110000是dtb，0×110000~0×610000是kernel。就是这中间0x10000大小的差距，产生了林林总总奇怪的现象，因此后期这部分还是有必要统一解决下的。要么都是一样的布局，要么用脚本自动屏蔽差异。